

FACULTY OF COMPUTING

Department of Information Technology

IT3130 – APPLICATION DEVELOPMENT

GROUP ASSIGNMENT • 30%

RideLink

Backend Microservices for a Ride-Sharing Platform

Module	IT3130 – Application Development
Assessment	Group Assignment
Group Size	Four students
Weight	30 marks out of the module total of 100
Release Date	10/09/2026
Submission Date	01/10/2026
Submission Method	Courseweb submission and scheduled demonstration/viva

Assessment scope

This is a backend-only assignment. Students are not required to develop a web or mobile frontend. Swagger UI/OpenAPI and a shared Postman collection are the official interfaces for development, testing and demonstration.

1. Assessment overview

This assignment requires each four-member group to design, implement, integrate and demonstrate a backend microservices solution for a fictional ride-sharing platform. The assessment emphasises architectural reasoning, context-appropriate interservice communication, application-development best practices, collaborative Git usage, quality assurance and continuous integration.

Final assignment mark: group component (15 marks) + individual component (15 marks) = 30 marks. *The individual component may differ among members of the same group.*

2. Module learning outcomes covered

LO	Module learning outcome	Evidence in this assignment
LO1	Recommend suitable software architectures with appropriate software technologies and frameworks to implement a software solution.	Architecture comparison and justification; microservice boundaries; framework selection; implementation of the assigned service.
LO2	Compare available inter-application communication methods.	Communication-interface design; interservice communication; comparison and justification of synchronous and asynchronous approaches.
LO3	Demonstrate the application of software engineering concepts and best practices.	SOLID principles, coding conventions, validation, security, testing, technical documentation and CI.
LO4	Employ suitable version-control strategies in software projects.	Branching strategy, pull requests, code review, meaningful commits and collaborative repository evidence.

3. Scenario: RideLink

RideLink is a fictional ride-sharing service inspired by common taxi-booking platforms. Passengers should be able to create accounts, request rides, receive fare estimates, obtain a suitable driver, track the ride lifecycle and record a simulated payment. Drivers should be able to maintain operational details, declare availability, accept an assigned request and update the ride status.

Your group has been appointed as the backend development team. Management requires a service-based architecture that supports independent development, clear business boundaries and future scaling. The system must therefore be decomposed into exactly four core microservices. Each student is the primary owner of one service, while the group remains jointly responsible for overall architecture and integration.

Fictional and simulated context

No real passengers, drivers, payments, maps or third-party production services are required. Use fictional test data. Locations may be represented using place names or simulated coordinates, and all payments must be simulated.

4. Required microservices and member ownership

#	Microservice	Primary owner	Minimum responsibility
1	Account Service	Member 1	Passenger and driver account registration; login and token issuance; role management; profile viewing and updating; account status management.
2	Driver & Vehicle Service	Member 2	Driver operational profile; vehicle details; availability status; service area; simulated current location; retrieval of eligible available drivers.
3	Ride Management Service	Member 3	Ride request creation; pickup and destination details; driver assignment; ride status lifecycle; acceptance, start, completion and cancellation; ride retrieval.

#	Microservice	Primary owner	Minimum responsibility
4	Fare & Payment Service	Member 4	Fare estimation; final fare calculation using a documented rule; simulated payment recording; payment status; receipt generation and retrieval.

The group may refine internal features with lecturer approval, but it must retain four meaningful business services of reasonably comparable complexity. An API Gateway, service registry or configuration server does not count as one of the four core services.

5. Minimum functional workflows

1. Account and access: register passenger and driver accounts, authenticate users and enforce appropriate roles.
2. Driver preparation: register vehicle/operational details, update availability and provide a simulated current location or service area.
3. Fare estimation: request an estimate for a pickup and destination using a clearly documented calculation rule.
4. Ride request and assignment: create a ride request, obtain eligible available drivers and assign or select a driver using a documented, simple approach.
5. Ride lifecycle: demonstrate requested, assigned, accepted, in-progress, completed and cancelled states with valid transition rules.
6. Completion and payment: calculate the final fare, record a simulated payment and retrieve a receipt/payment record.
7. Negative scenarios: demonstrate at least two failures such as no available driver, invalid status transition, unauthorised access, invalid input or failed simulated payment.

6. Technical and engineering requirements

6.1 Architecture and data ownership

- Implement four independently executable backend services using the language and framework prescribed or approved during laboratory sessions.
- Each service must own its persistence boundary. One service must not directly query or modify another service's database or tables.
- Document service responsibilities, dependencies and data ownership through an architecture diagram and at least one sequence diagram.
- Compare the selected microservices approach with a monolithic alternative and justify its suitability and limitations for this scenario.

6.2 APIs and communication

- Expose RESTful JSON APIs for externally exercised operations using appropriate resource names, HTTP methods, status codes, validation and consistent error responses.
- For each interservice interaction, select and justify a communication interface that suits its context. Choices may include synchronous REST or gRPC, asynchronous messaging through a queue, or an approved equivalent; different interactions may use different approaches.
- Demonstrate at least two meaningful interservice interactions within the required workflows using the selected approach or approaches.
- Use stable identifiers and avoid direct cross-service database access.

- In the report, compare the selected communication approach with at least one relevant alternative and explain why each implemented choice is appropriate for its workflow.
- Provide accurate OpenAPI/Swagger documentation for REST endpoints and suitable contracts or configuration evidence for any gRPC or asynchronous messaging interfaces.

6.3 Security and software quality

- Implement authentication and role-based authorisation for relevant passenger, driver and administrative operations.
- Do not commit passwords, tokens, connection strings or other secrets to the repository. Use environment variables or an approved configuration approach.
- Apply appropriate SOLID principles, coding conventions, meaningful naming, validation, exception handling and technical documentation.
- Each service must contain meaningful unit tests. The integrated solution must include tests or repeatable Postman scenarios covering successful and negative workflows.

6.4 Version control and continuous integration

- Maintain the complete project in one shared Git repository accessible to the teaching team.
- Agree and document a suitable branching workflow. Use feature branches, meaningful commits, pull requests and peer review before integration.
- Contribution is judged by quality, continuity and traceability—not by commit count alone.
- Configure a CI pipeline that automatically builds and runs tests for all four services on the agreed repository events.
- The main branch must represent an integrated and demonstrable version of the system at submission time.

6.5 Official API client and scope boundaries

No frontend development is required

Use Swagger UI/OpenAPI and a shared Postman collection to exercise and demonstrate the system. A web or mobile frontend, Docker deployment, cloud hosting, live maps and real payment integration are optional enhancements and receive no separate marks unless later announced in writing.

7. Group organisation and individual accountability

- Record the primary owner of each microservice in the report and repository README.
- All members must participate in architecture, API-contract and integration decisions, even though each member owns one service.
- Each member must contribute traceable code, tests, documentation and review activity through their own repository identity.
- Every member must be able to run and explain the integrated system, not only their assigned service.
- Group marks are normally shared. Individual marks are awarded separately using service evidence, Git history, testing, documentation and the viva.
- If authorship or contribution cannot be verified, the relevant individual marks may be reduced or withheld in accordance with institute regulations.

8. Required deliverables

Source repository: Complete history, four services, tests, CI configuration and a release tag identifying the assessed version.

Technical report: One PDF, recommended 8–12 pages excluding appendices, containing architecture rationale, service/data boundaries, framework choice, diagrams, communication-method comparison, security/testing approach, CI evidence, limitations and individual contribution statement.

Repository README: Prerequisites, configuration, start-up order, commands, test instructions, endpoint locations and sample credentials/test data.

API and interface evidence: OpenAPI/Swagger documentation for REST endpoints, an exported Postman collection and environment using non-sensitive example values, plus suitable contracts or configuration evidence for any gRPC or asynchronous messaging interfaces.

Testing evidence: Unit-test results for every service and evidence of integrated successful and negative workflows.

Demonstration and viva: A scheduled group demonstration followed by individual questioning. Exact duration and schedule will be announced.

9. Submission instructions

- Submit one group package through Courseweb by the announced deadline. The nominated group member may submit on behalf of the group, but all member details must be included.
- Recommended file name: IT3130_GroupXX_RideLink.zip. The package should contain the PDF report, Postman files and any required supporting evidence. Include the private repository link and grant access to the teaching team before the deadline.
- Do not include dependency folders, build outputs, database passwords, access tokens or unnecessary binary files.
- The assessed version is the repository release/tag and submitted package available at the deadline. Changes made afterwards will not normally be considered.
- Late submission, extensions and special consideration are governed by the current institute regulations and module announcements.

10. Marking scheme

The assignment contributes 30 marks to the module total. The first 15 marks evaluate the integrated group solution; the remaining 15 marks evaluate each member individually.

Code	Criterion	Component	LO mapping	Marks
G1	Architecture and service decomposition	Group	LO1	3
G2	Integrated business workflows	Group	LO1-LO3	4
G3	Communication-interface design and interservice communication	Group	LO2	3
G4	Shared engineering quality, security and CI	Group	LO3-LO4	3
G5	Technical documentation and group demonstration	Group	LO1-LO3	2
I1	Assigned microservice functionality and ownership	Individual	LO1-LO3	6
I2	Individual code quality, testing and API documentation	Individual	LO3	3
I3	Git contribution and version-control practice	Individual	LO4	3
I4	Individual viva and reflection	Individual	LO1-LO4	3
TOTAL				30

Individual mark variation

Members of the same group may receive different final assignment marks. A working group system does not replace the requirement for verifiable individual implementation, testing, Git evidence and understanding.

11. Detailed marking rubric

Assessors may award any value within the stated band according to the quality and completeness of the evidence. Critical academic-integrity or non-submission cases are handled under institute regulations rather than through rubric averaging.

G1 • Architecture and service decomposition | Group | 3 marks | LO1

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	Four cohesive business services with clear boundaries and independent data ownership. Architecture and framework choices are convincingly justified; diagrams accurately show responsibilities and dependencies with low coupling.
Very good (70–84%) 2.10–2.54	Clear four-service decomposition, data ownership and sound architectural rationale. Diagrams are accurate, with only minor boundary or dependency issues.
Competent (60–69%) 1.80–2.09	Workable decomposition and basic justification. Most boundaries are meaningful, although some responsibilities or dependencies are uneven.
Basic (50–59%) 1.50–1.79	Four services are present, but boundaries are partly arbitrary or tightly coupled. Justification and diagrams are limited.
Inadequate (0–49%) 0.00–1.49	The solution is substantially monolithic, consists of disconnected APIs, or lacks understandable boundaries, ownership and architectural justification.

G2 • Integrated business workflows | Group | 4 marks | LO1–LO3

Performance level and score	Descriptor
Excellent (85–100%) 3.40–4.00	Required end-to-end workflows operate reliably across services, including successful and negative scenarios. Ride lifecycle and shared identifiers remain consistent; service failures and invalid state transitions are handled gracefully.
Very good (70–84%) 2.80–3.39	All major workflows operate across services with correct lifecycle handling. Minor integration defects or limited negative-case handling are present.
Competent (60–69%) 2.40–2.79	Core ride workflow operates, but one required workflow or several edge cases are incomplete. Integration is generally understandable.
Basic (50–59%) 2.00–2.39	Only a partial workflow operates; substantial manual intervention, hard-coded data or inconsistent states are evident.
Inadequate (0–49%) 0.00–1.99	Services do not form a working integrated system, or the principal ride-booking workflow cannot be demonstrated.

G3 • Communication-interface design and interservice communication | Group | 3 marks | LO2

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	The group selects context-appropriate synchronous and/or asynchronous interfaces, justifies each choice and implements them correctly. At least two meaningful interservice interactions are demonstrated with suitable contracts, validation, error handling and documentation.
Very good (70–84%) 2.10–2.54	Appropriate communication interfaces and meaningful service interactions are implemented and justified, with only minor contract, documentation or error-handling issues.
Competent (60–69%) 1.80–2.09	Services communicate through generally suitable interfaces, but the justification, contracts, validation, error handling or comparison of approaches has several weaknesses.
Basic (50–59%) 1.50–1.79	Basic interservice communication exists, but interface choices are weakly justified, fragile, poorly documented or largely simulated.
Inadequate (0–49%) 0.00–1.49	Interservice communication is absent or unsuitable, or the group cannot provide a defensible, context-based justification for its interface choices.

G4 • Shared engineering quality, security and CI | Group | 3 marks | LO3–LO4

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	Role-based authentication/authorization, input validation, safe secret handling and consistent error practices are applied across the system. The CI pipeline reproducibly builds and tests all four services on the agreed repository events.
Very good (70–84%) 2.10–2.54	Security, validation and error-handling practices are consistently applied. CI builds and tests all services, with only minor configuration or evidence gaps.
Competent (60–69%) 1.80–2.09	Essential security and validation are present, and CI runs for most services; implementation is somewhat inconsistent or incomplete.
Basic (50–59%) 1.50–1.79	Security is minimal or inconsistently applied; secrets/configuration practices require improvement. CI is partial, unreliable or limited to compilation.
Inadequate (0–49%) 0.00–1.49	Major security risks, committed secrets, missing validation or no functioning CI pipeline are evident.

G5 • Technical documentation and group demonstration | Group | 2 marks | LO1–LO3

Performance level and score	Descriptor
Excellent (85–100%) 1.70–2.00	README, architecture/sequence diagrams, setup instructions, OpenAPI output and Postman collection are complete, accurate and reproducible. The demonstration clearly proves all required workflows and constraints.
Very good (70–84%) 1.40–1.69	Documentation and demonstration are complete and mostly reproducible, with only minor omissions or inaccuracies.
Competent (60–69%) 1.20–1.39	Core documentation and demonstration are understandable, but several setup, diagram, API or evidence details are missing.
Basic (50–59%) 1.00–1.19	Documentation is limited and reproduction requires substantial assistance; the demonstration covers only basic functionality.
Inadequate (0–49%) 0.00–0.99	Documentation/evidence is missing or misleading, or the system cannot be reasonably reproduced and demonstrated.

I1 • Assigned microservice functionality and ownership | Individual | 6 marks | LO1–LO3

Performance level and score	Descriptor
Excellent (85–100%) 5.10–6.00	The assigned service fully implements its required business capabilities, persistence, validation and service interactions. The student demonstrates clear ownership and can trace design decisions to working code and system behavior.
Very good (70–84%) 4.20–5.09	The assigned service is substantially complete and integrated, with minor defects. The student demonstrates strong ownership and understanding.
Competent (60–69%) 3.60–4.19	Core service functions operate, but some rules, endpoints, validation, persistence or integrations are incomplete. Ownership is adequately demonstrated.
Basic (50–59%) 3.00–3.59	Only basic service functions operate; implementation is incomplete, fragile or heavily dependent on others. Ownership evidence is limited.
Inadequate (0–49%) 0.00–2.99	The assigned service is largely non-functional, copied without demonstrated understanding, or the student cannot establish meaningful ownership.

I2 • Individual code quality, testing and API documentation | Individual | 3 marks | LO3

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	Code consistently demonstrates suitable structure, naming, SOLID principles and maintainability. Meaningful unit tests cover normal, boundary and failure behavior; service-level OpenAPI documentation is accurate and complete.
Very good (70–84%) 2.10–2.54	Code quality and maintainability are strong. Tests and API documentation cover the important behavior, with minor gaps.
Competent (60–69%) 1.80–2.09	Code is generally understandable and includes useful tests/documentation, but design, coverage, edge cases or consistency require improvement.
Basic (50–59%) 1.50–1.79	Code quality is inconsistent; tests are superficial or limited to happy paths; API documentation is incomplete.
Inadequate (0–49%) 0.00–1.49	Code is difficult to maintain, tests are absent/ineffective, or the student's service APIs are undocumented.

I3 • Git contribution and version-control practice | Individual | 3 marks | LO4

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	A sustained sequence of meaningful commits, branches, pull requests and reviews provides clear evidence of individual contribution. Changes are well described, appropriately scoped and integrated through the agreed workflow.
Very good (70–84%) 2.10–2.54	Strong, traceable contribution through the agreed workflow, with only minor weaknesses in commit scope, review evidence or branch discipline.
Competent (60–69%) 1.80–2.09	Adequate contribution is visible, but work is uneven, some commits are overly large, or pull-request/review evidence is limited.
Basic (50–59%) 1.50–1.79	Contribution is concentrated near the deadline, difficult to trace, or largely consists of bulk commits and direct main-branch changes.
Inadequate (0–49%) 0.00–1.49	Little or no verifiable contribution exists, repository history is manipulated/unclear, or the agreed version-control workflow was not used.

I4 • Individual viva and reflection | Individual | 3 marks | LO1–LO4

Performance level and score	Descriptor
Excellent (85–100%) 2.55–3.00	The student accurately explains the overall architecture, own code, APIs, communication, tests and Git evidence; diagnoses questions confidently and reflects critically on trade-offs and improvements.
Very good (70–84%) 2.10–2.54	The student explains the system and own contribution accurately, with sound responses and useful reflection; only minor knowledge gaps are evident.
Competent (60–69%) 1.80–2.09	The student explains core concepts and own work, but has gaps regarding integration, design decisions, testing or version-control evidence.
Basic (50–59%) 1.50–1.79	The explanation is superficial or inconsistent with the submitted work; substantial prompting is required.
Inadequate (0–49%) 0.00–1.49	The student cannot explain or demonstrate the submitted contribution, or is absent without an approved alternative arrangement.

12. Academic integrity and responsible tool use

- All submitted work must be produced by the group and understood by the responsible members. Code, diagrams, text or configurations obtained from external sources must be appropriately acknowledged.
- The use of generative-AI tools must follow the institute's current academic-integrity policy. Any permitted use should be declared in an appendix, including the purpose for which the tool was used.
- Every student remains responsible for the correctness, security and explainability of submitted work, regardless of the tools used during development.
- Repository history, viva responses and implementation evidence may be used to verify authorship and contribution. Plagiarism, impersonation, fabricated evidence or unauthorised collaboration will be handled under institute regulations.

End of assignment brief